



Subinterpreters and GIL-less Python

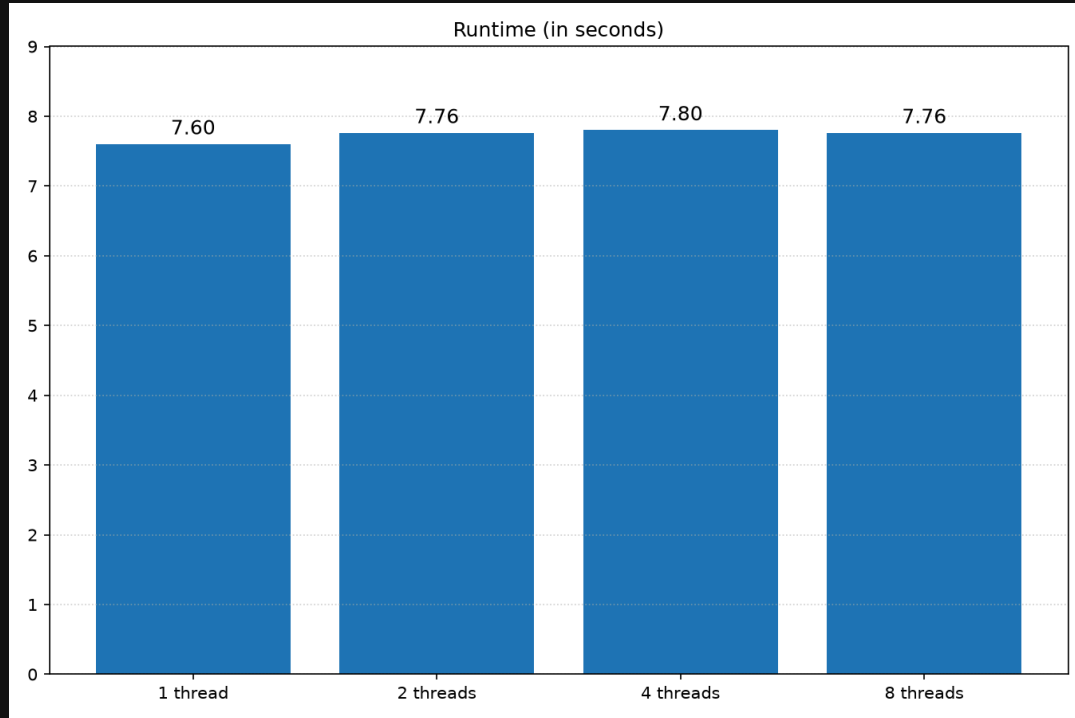
18 June 2026, 19:00h
@ SiteGround HQ



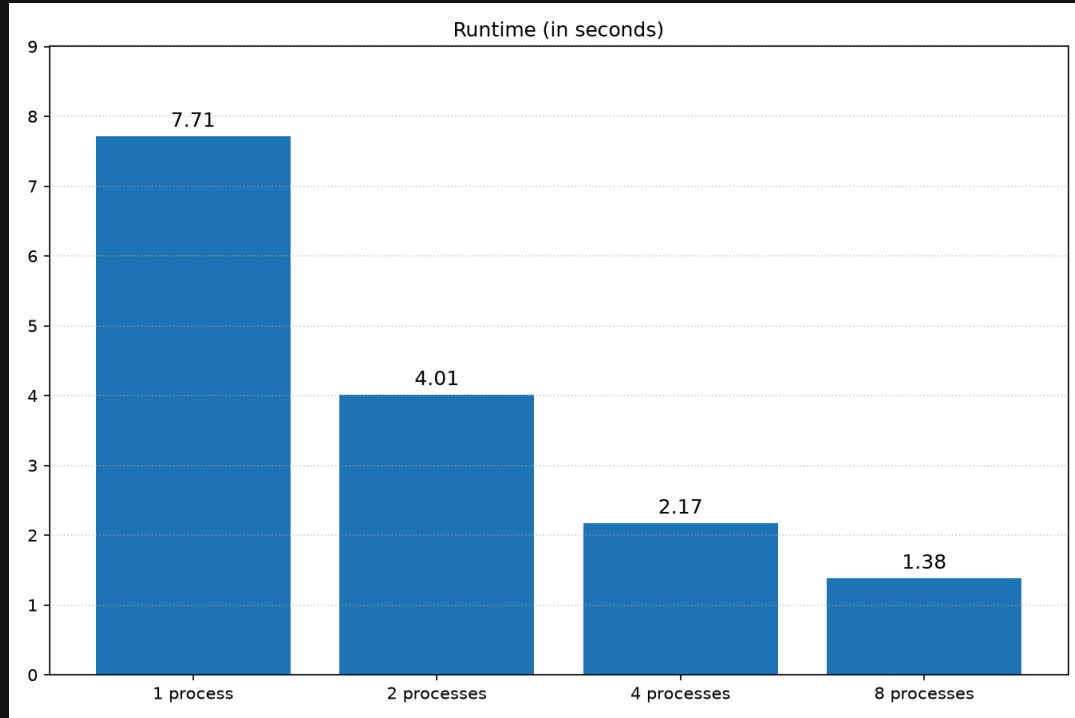
Lyuboslav Karev
AI Engineer @ SiteGround



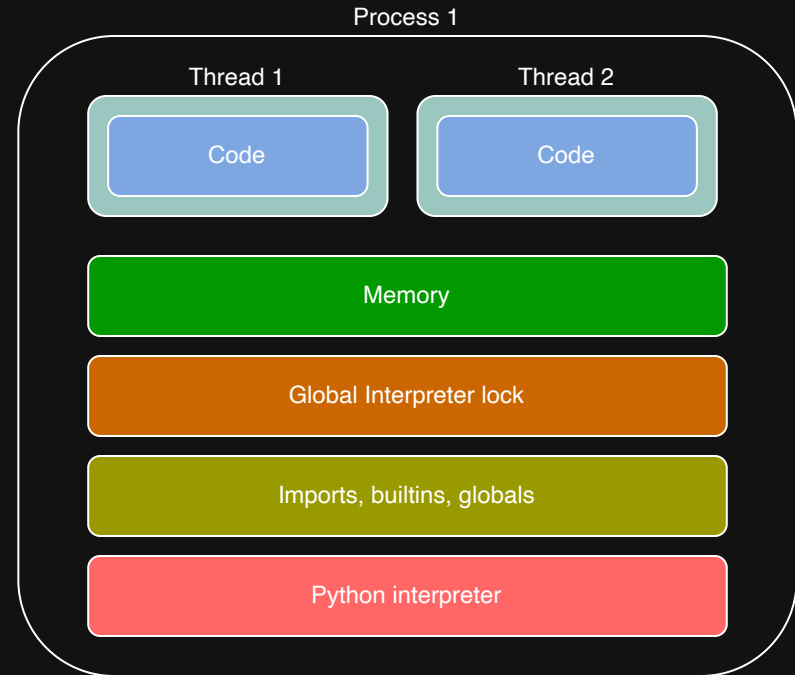
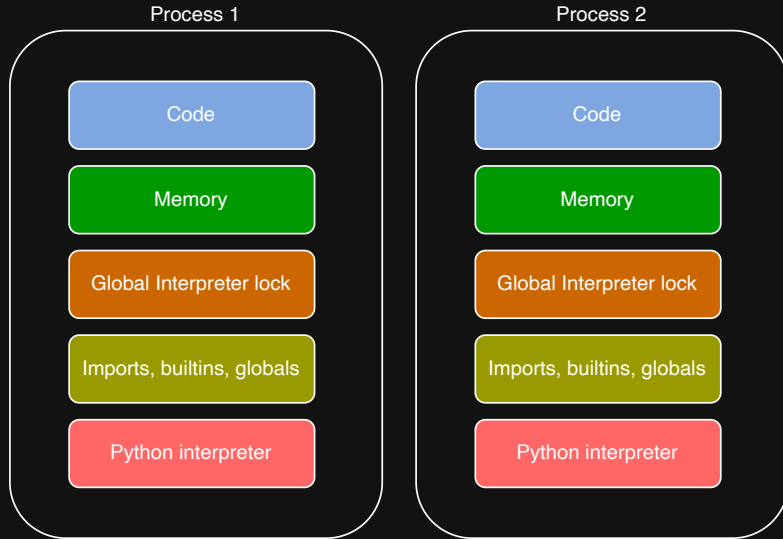
Threads



vs processes ?



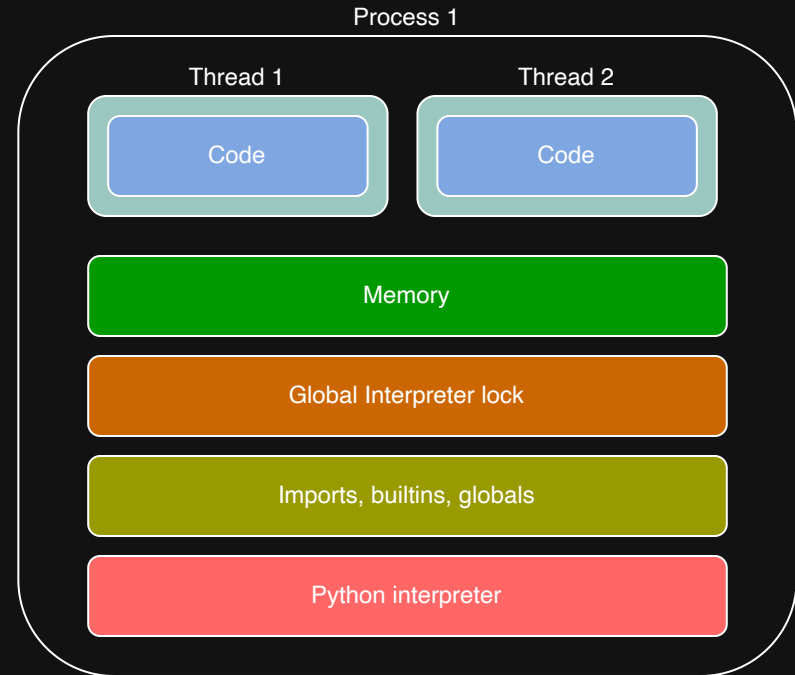
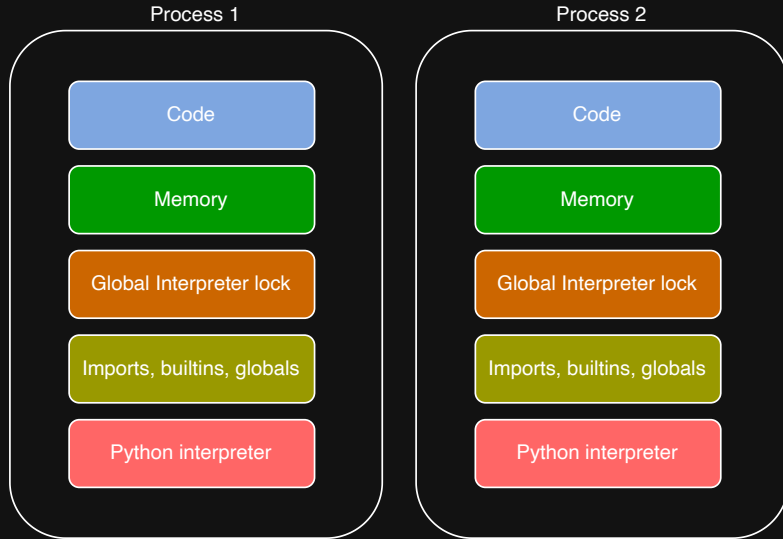
Processes vs threads



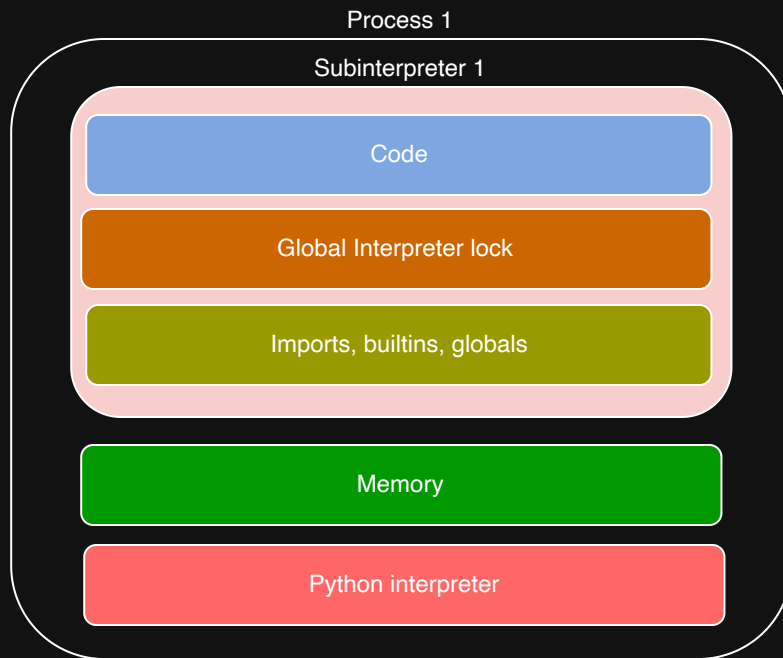
What is the GIL?

- The **Global Interpreter Lock** is a mutex that protects access to Python objects.
- It prevents multiple native threads from executing Python bytecodes simultaneously.
- Without the GIL, reference counting (CPython's memory management) would require locks on every object.
- The GIL is released during I/O operations, allowing some concurrency.
- It was introduced to simplify CPython's implementation and integration with C extensions.

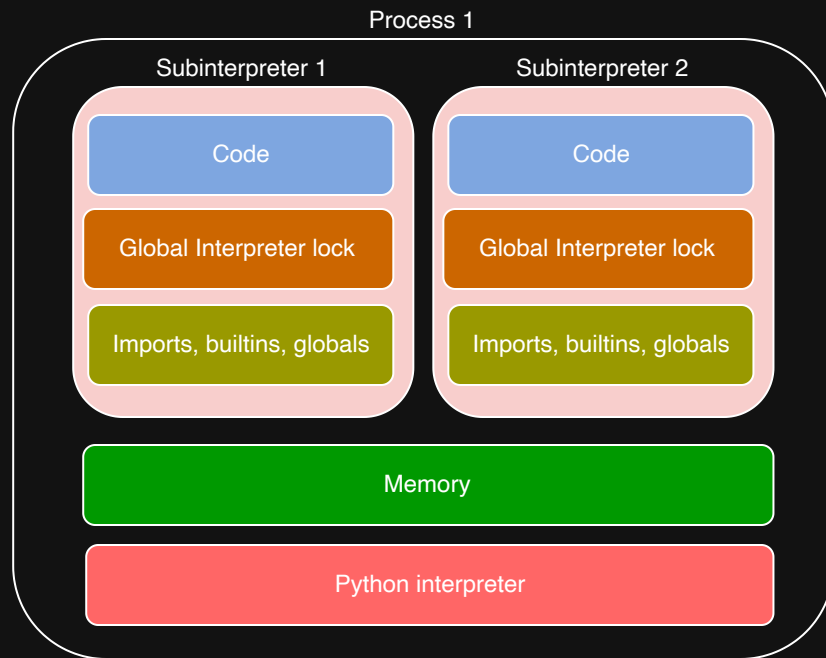
Processes vs threads



What is a subinterpreter?



What is a subinterpreter ?



Example 01 - `Interpreter`

```
1  from concurrent import interpreters
2
3  interpreter = interpreters.create()
4
5  interpreter.exec("print('Hello from a different interpreter')")
6
7  def answer(a, b) → int:
8      return a + b
9
10 result = interpreter.call(answer, 2, 3)
11
12 print(result)
13
14 interpreter.close()
```

```
$ uv run 03_interpreter.py
Hello from a different interpreter
5
```

Example 02 - `prepare_main``

```
1  from concurrent import interpreters
2
3  interpreter = interpreters.create()
4
5  interpreter.prepare_main({"a": 42}) # Comment this line for error
6
7  interpreter.exec("print(a)")
8
9  interpreter.close()
```

```
$ uv run 04_prepare_main.py
42
```

```
$ uv run 04_prepare_main.py
concurrent.interpreters.ExecutionFailed: NameError: name 'a' is not defined
```

Example 03 - Communication between subinterpreters

```
1  from concurrent import interpreters
2
3
4  def worker(queue):
5      import math
6
7      numbers = [112272535095293, 112582705942171, 115280095190773]
8
9      for n in numbers:
10         sqrt_n = int(math.floor(math.sqrt(n)))
11         result = n > 1 and all(n % i != 0 for i in range(2, sqrt_n + 1))
12         queue.put((n, result))
13
14
15  queue = interpreters.create_queue()
16  interpreter = interpreters.create()
17
```

```
uv run 06_interpreter_communication.py
112272535095293: prime
112582705942171: prime
115280095190773: prime
```

Example 03 - Communication between subinterpreters

```
9     for n in numbers:
10         sqrt_n = int(math.floor(math.sqrt(n)))
11         result = n > 1 and all(n % i != 0 for i in range(2, sqrt_n + 1))
12         queue.put((n, result))
13
14
15 queue = interpreters.create_queue()
16 interpreter = interpreters.create()
17
18 interpreter.call(worker, queue)
19
20 for _ in range(3):
21     number, is_prime = queue.get()
22     print(f"{number}: {'prime' if is_prime else 'not prime'}")
23
24 interpreter.close()
25
```

```
uv run 06_interpreter_communication.py
```

```
112272535095293: prime
```

```
112582705942171: prime
```

```
115280095190773: prime
```

Example 03 - Communication between subinterpreters

```
1  from concurrent import interpreters
2
3
4  def worker(queue):
5      import math
6
7      numbers = [112272535095293, 112582705942171, 115280095190773]
8
9      for n in numbers:
10         sqrt_n = int(math.floor(math.sqrt(n)))
11         result = n > 1 and all(n % i != 0 for i in range(2, sqrt_n + 1))
12         queue.put((n, result))
13
14
15  queue = interpreters.create_queue()
16  interpreter = interpreters.create()
17
```

```
uv run 06_interpreter_communication.py
112272535095293: prime
112582705942171: prime
115280095190773: prime
```

Sharing objects

- Subinterpreters **cannot** effectively share objects between themselves.
- Functions are **not shareable** when they have dependencies outside the new interpreter.
- The immutable objects are shared directly (or copied in an efficient manner):
 - ``None``
 - ``bool``
 - ``bytes``
 - ``float``
 - ``int``
 - ``str``
 - ``tuple``
- Copying with ``pickle`` is used for the other objects

Example 04 - numpy in subinterpreter

```
1  from concurrent import interpreters
2  import numpy as np
3
4  interpreter = interpreters.create()
5
6  def compute_mean(arr):
7      return arr.mean()
8
9  data = np.random.rand(1_000_000)
10 interpreter.call(compute_mean, data)
```

```
ImportError: module numpy._core._multiarray_umath does not support loading in subinterpreters
```

```
...
```

```
Importing the numpy C-extensions failed. This error can happen for
many reasons, often due to issues with your setup or how NumPy was
installed.
```

```
...
```

```
concurrent.interpreters.NotShareableError: object could not be unpickled
```

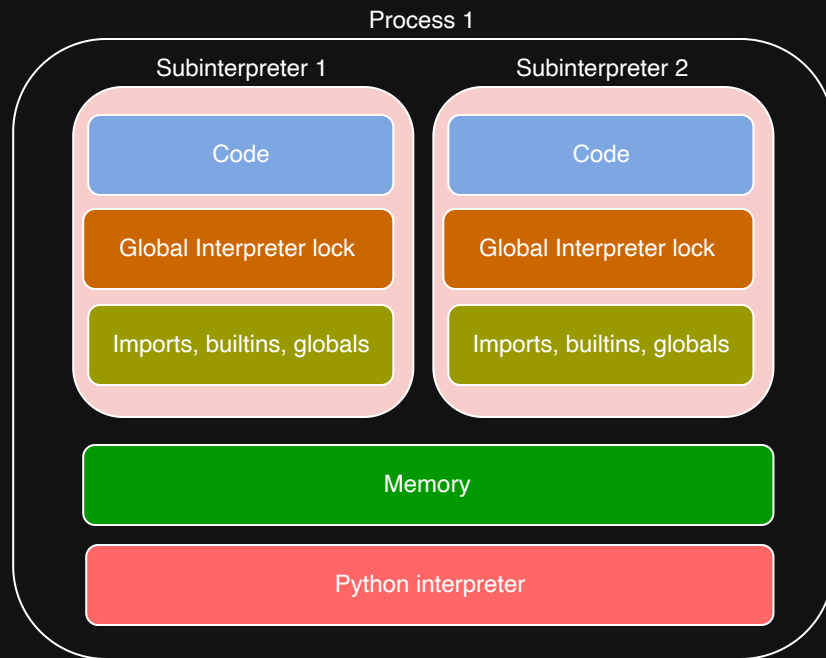
```
...
```

```
concurrent.interpreters.NotShareableError: args not shareable
```

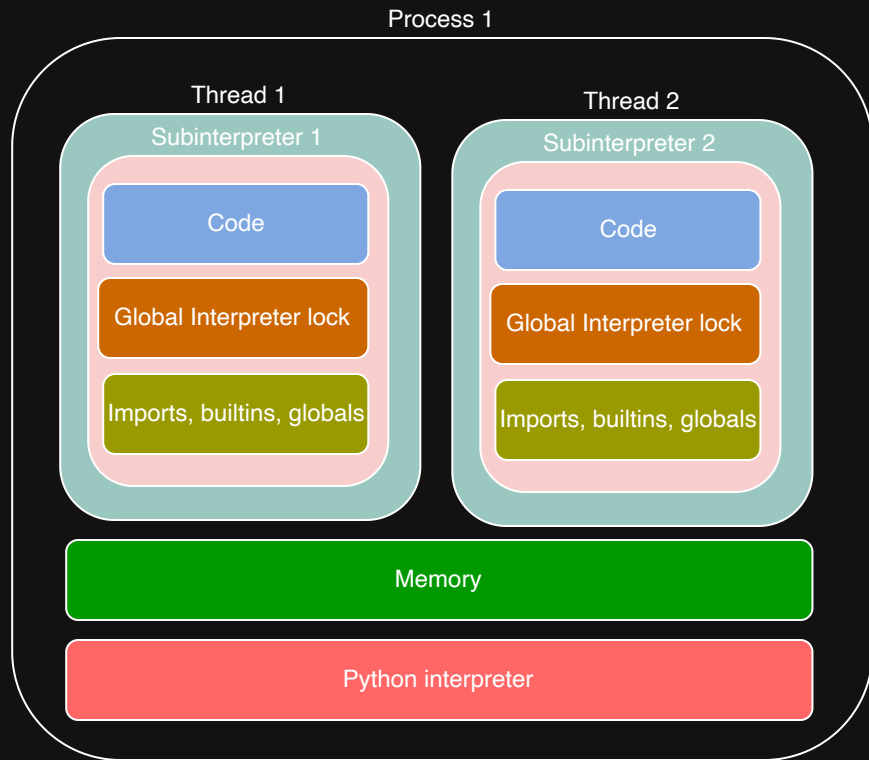
C extensions & subinterpreters

- C extensions must explicitly declare support via `Py_MOD_PER_INTERPRETER_GIL_SUPPORTED`.
- Without the declaration, importing raises a hard `ImportError`
- Notable packages **not yet supported**:
 - `numpy` - `InterpreterPoolExecutor` with arrays fails entirely
 - `torch` - blocked because `_ctypes` itself doesn't declare support
 - Cython-compiled extensions (e.g. `pyarrow`, `spacy`)

What is a subinterpreter ?



What is a subinterpreter ?



Example 05 - `call_in_thread`

```
1  from concurrent import interpreters
2
3
4  def sleepy_print():
5      print("Hello from the subinterpreter!")
6      import time
7
8      time.sleep(2)
9
10
11  interpreter = interpreters.create()
12
13  t = interpreter.call_in_thread(sleepy_print)
14
15  import time
16  time.sleep(2)
17
```

```
$ time uv run 05_call_in_thread.py
Hello from the subinterpreter!
Hello from the main interpreter!
uv run 05_call_in_thread.py 0.04s user 0.03s system 3% cpu 2.162 total
```

Example 06 - Baseline

```
1  import math
2
3  NUMBERS = [
4      112272535095293,
5      112582705942171,
6      112272535095293,
7      115280095190773,
8      115797848077099
9  ] * 10
10
11  def is_prime(n):
12      if n < 2:
13          return False
14      if n == 2:
15          return True
16      if n % 2 == 0:
17          return False
18
19      sqrt_n = int(math.floor(math.sqrt(n)))
20      for i in range(3, sqrt_n + 1, 2):
21          if n % i == 0:
22              return False
23      return True
```

Example 07 - Multiple processes

```
1  import math
2  import sys
3  import time
4
5  from concurrent.futures import ProcessPoolExecutor
6
7  ...
8
9  if __name__ == "__main__":
10     print(sys.version)
11     start = time.time()
12
13     workers = 4
14
15     with ProcessPoolExecutor(max_workers=workers) as executor:
16         executor.map(is_prime, NUMBERS)
17
18     print(f"{workers} workers took {time.time() - start:.2f} seconds")
19
```

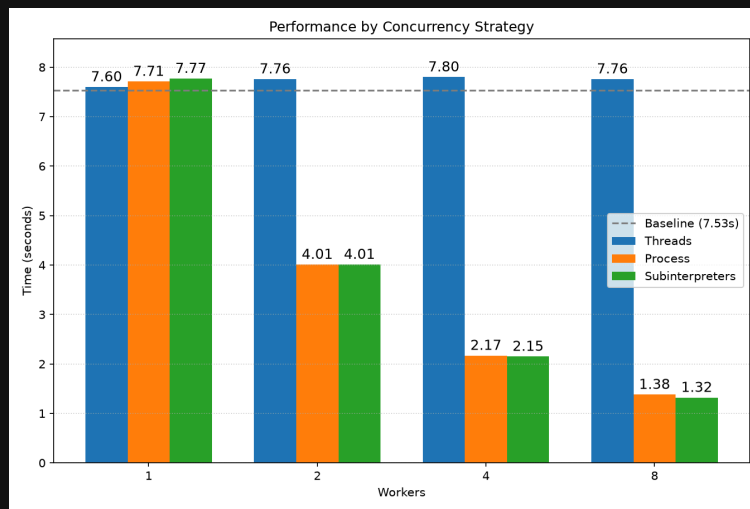
Example 08 - Multiple threads

```
1  import math
2  import sys
3  import time
4
5  from concurrent.futures import ThreadPoolExecutor
6
7  ...
8
9  if __name__ == "__main__":
10     print(sys.version)
11     start = time.time()
12
13     workers = 4
14
15     with ThreadPoolExecutor(max_workers=workers) as executor:
16         executor.map(is_prime, NUMBERS)
17
18     print(f"{workers} workers took {time.time() - start:.2f} seconds")
19
```

Example 09 - Multiple subinterpreters

```
1  import math
2  import sys
3  import time
4
5  from concurrent.futures import InterpreterPoolExecutor
6
7  ...
8
9  if __name__ == "__main__":
10     print(sys.version)
11     start = time.time()
12
13     workers = 4
14
15     with InterpreterPoolExecutor(max_workers=workers) as executor:
16         executor.map(is_prime, NUMBERS)
17
18     print(f"{workers} workers took {time.time() - start:.2f} seconds")
19
```

Performance comparison



So, subinterpreters are the answer ?

Pros

True parallelism, bypassing the GIL

Low overhead compared to processes

State isolation

Cons

Object sharing can be hard

Higher overhead than threads

Library support is low

GIL-less (Freethreaded) Python

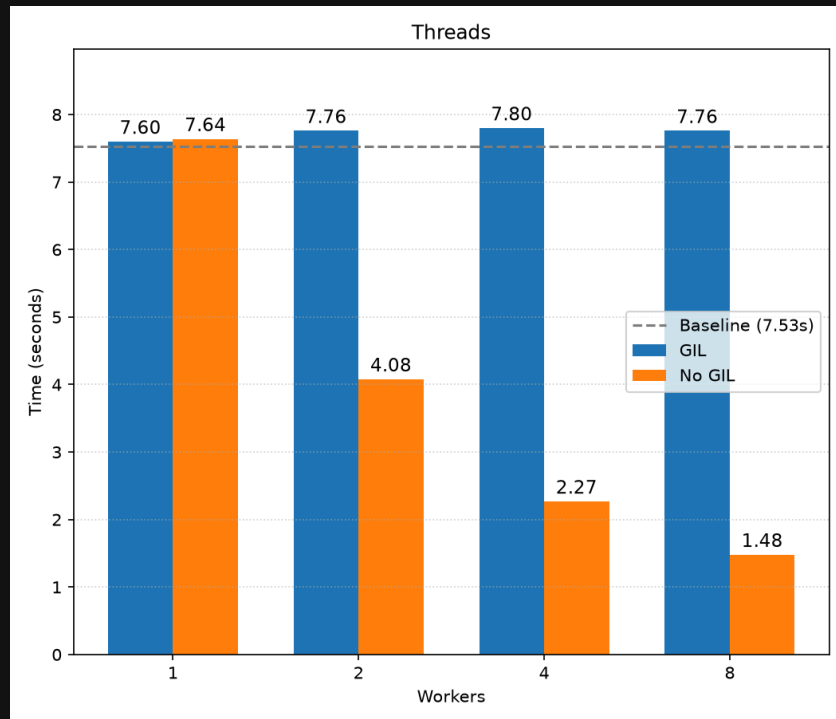
Freethreaded Python

- Freethreaded Python removes the GIL entirely (PEP 703).
- First introduced in Python3.13, more stable in Python3.14
- Consequences to be aware of:
 - **Thread safety** - objects that were previously safe due to the GIL may no longer be.
 - **Immortalization** - some objects are made immortal to avoid reference-count contention.
 - **Iterators** - may not be thread-safe without explicit locking.
 - **Single-threaded performance** - slightly slower due to per-object locking overhead.

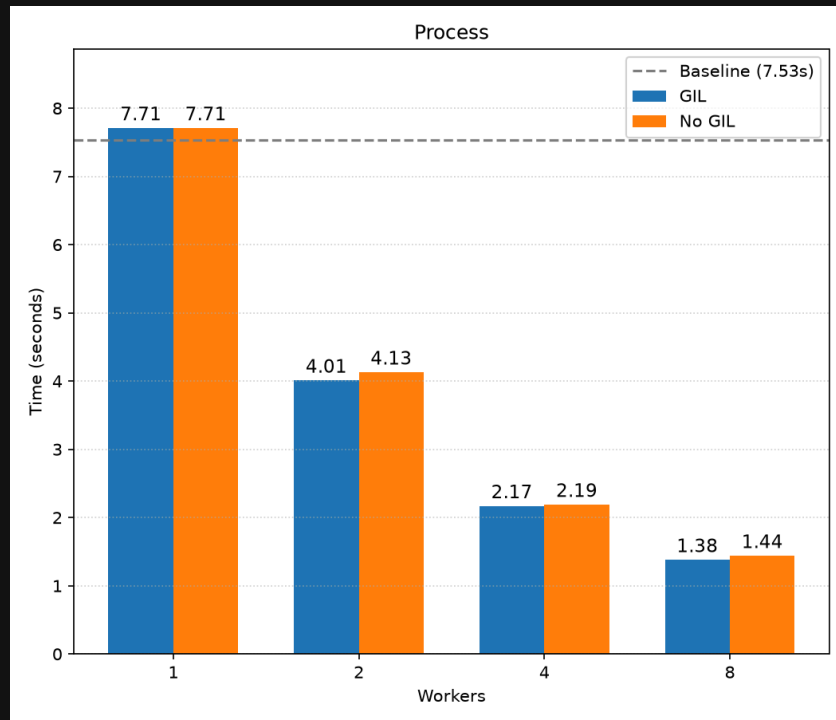
Setting up freethreaded Python

- When using `uv``, setting up a freethreaded version of Python is simple:
 - `uv python install 3.14t``, followed by `uv python pin 3.14t``. (To revert back: `uv python pin 3.14``)
- When using the installer, when customizing the installation, use the "Free-threaded Python" option

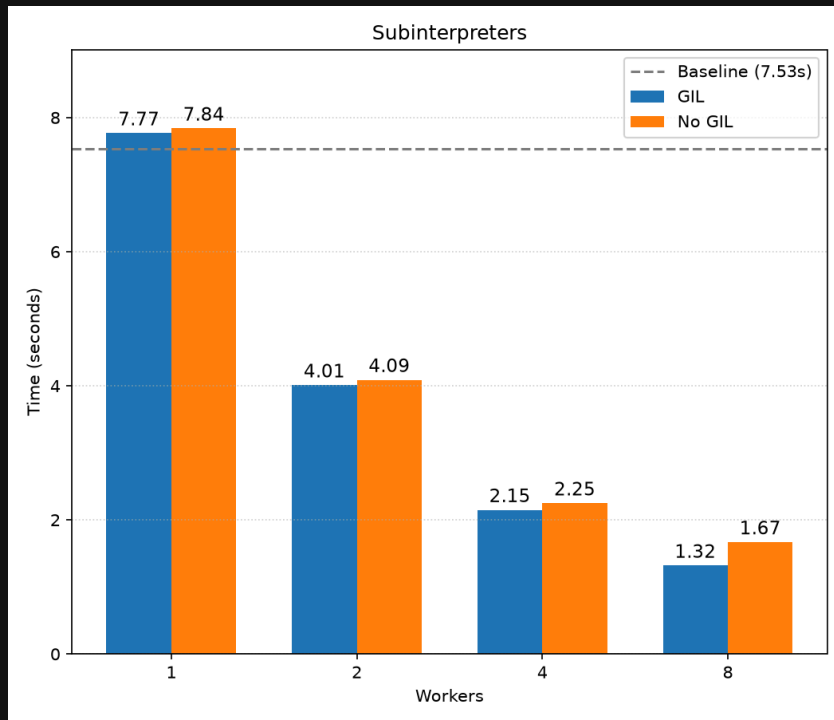
Benchmark with freethreaded Python - Threads



Benchmark with freethreaded Python - Processes



Benchmark with freethreaded Python - Subinterpreters



Library support

Library	Free-Thread Readiness	Key Opportunity	Key Risk
Django	Low - no thread-safety audit done yet	CPU-bound views gain true parallelism; async views benefit more	ORM <code>save()</code> races, lazy translation state, thread-local DB connections under higher concurrency
Gunicorn	Low - no targeted support yet	<code>gthread`</code> workers could gain true CPU parallelism	Gains not materializing at thread level yet; use workers instead
NumPy	Medium-High - experimental support since 2.1	Parallelizing Python-side orchestration code	Mutating shared arrays can crash the interpreter; <code>dtype=object`</code> is especially dangerous
PyTorch	Medium - 3.13t wheels available	Parallel inference, fixing <code>DataParallel`</code> , threaded <code>DataLoader`</code>	<code>torch.compile`</code> is broken; dependency chain (PyO3/Cython) problematic
psycopg2-binary	None - not yet supported	-	-
psycopg-binary	None - not yet supported	-	-
tokenizers	None - not yet supported	-	-
opencv-python	None - not yet supported	-	-
spacy	None - not yet supported	-	-

Summary

- **Subinterpreters** offer in-process isolation with a per-interpreter GIL and sit somewhere between threads and processes. Overall they seem like the stop-gap solution until the free-threaded Python becomes mainstream.
- **Freethreaded** Python removes the GIL and enables threads to scale for CPU-bound work. However the ecosystem support is still catching up. While introduced in Python3.13, the plan for making the free-threaded build the default has no set timeline.